UdeSA-migos

Introducción **3**
Requisitos **4**
Inteligencia Artificial **5**
Condición de aprobación **5**
Epicas **6**
E.1 Usuarios 6
H1. Registro de Usuarios ⚡ 6
H2. Inicio de Sesión ⚡ 6
H3. Cierre de Sesión ⚡ 7
H4. Eliminación de Cuenta ⚡ 7
H5. Olvidé Mi Contraseña ⚡ 8
H6. Editar mi perfil ⚡ 9
H7. Preferencias ⚡ 9
H8. Foto de Perfil ➕ 10
H9. Social Login ➕ 10
H10. Tema de la Aplicación ➕ 11
H11. Enviar Feedback o Reportar Error ➕ 11
H12. Aceptación de Términos y Política de Privacidad ⚡ 12
H13. Cambiar Contraseña ⚡ 12
H14. Onboarding de Permisos Móviles ➕ 13
E.2 Amistades 14
H1. Agregar nuevo amigo ⚡ 14
H2. Listado de Solicitudes Pendientes ⚡ 14
H3. Eliminar amigo ⚡ 15
H4. Modo Fantasma ➕ 15
H5. Modo Privado ⚡ 16
H6. Descubrimiento ⚡ 16
H7. Visualización de Lista de Amigos ⚡ 17
H8. Bloquear Usuario ⚡ 17
H9. Denunciar Usuario ➕ 17
H10. Mensajes entre Amigos ➕ 18
H11. Amigos en línea ➕ 18
H12. Lista de mejores amigos ➕ 19
H13. Invitar Usuarios Externos ➕ 19
H14. Visualización de Perfil de Amigo ⚡ 20

E.3 Ubicaciones 20
H1. Enviar Ubicación ⚡ 20
H2. Listado de Ubicaciones ⚡ 21
H3. Visualización en Mapa Interactivo ⚡ 21
H4. Sincronización Offline ➕ 21
H5. Establecer Punto de Encuentro ➕ 22
H6. Historial de Ubicaciones➕ 22
H7. Etiqueta de Lugar Manual ⚡ 23
H8. "Me gusta" a la Ubicación ➕ 23
H9. Color del Pin en el Mapa ➕ 24
H10. Zonas Seguras / Zonas de Privacidad ➕ 24
E.4 Notificaciones 25
H1. Recibir notificaciones de amistad ⚡ 25
H2. Notificación de solicitud aceptada ⚡ 25
H3. Centro de Notificaciones In-App ⚡ 25
H4. Notificación por proximidad ➕ 26
H5. Configurar Preferencias de Notificación ➕ 26
E.5 Administradores (Backoffice Web) 27
H1. Creación de Administradores ⚡ 27
H2. Inicio de sesión como Administrador ⚡ 27
H3. Visualización de Métricas ⚡ 28
H4. Buscador y Detalles de Usuarios ⚡ 28
H5. Bloqueo de Usuarios por Admin ⚡ 28
H6. Logs de Auditoría ➕ 29
H7. Gestión de Denuncias ➕ 29
H8. Exportar Datos ➕ 29
H9. Registro de Última Conexión ⚡ 30
H10. Borrado Forzado de Textos ➕ 30
H11. Estado de los Microservicios ⚡ 30
H12. Gestión de Feedback/Reportes➕ 31
Puntos de Historias de Usuario **32**
Contrato Moral **34**

Introducción

En esta oportunidad vamos a desarrollar una aplicación mobile **UdeSA-migos**, que permite a los estudiantes compartir su ubicación en tiempo real con sus amigos de la Universidad.

Adicionalmente, se requiere una segunda aplicación que funcione como **backoffice**, destinada exclusivamente a administradores. Su objetivo será monitorear el funcionamiento de la aplicación principal, detectar posibles fallas, identificar anomalías y garantizar la correcta operación del sistema.

Para llevar adelante el desarrollo, el team leader definió un conjunto de requisitos funcionales y no funcionales que debemos considerar desde el inicio del proyecto.

Las **historias de usuario constituyen la base del alcance del sistema**: a partir de ellas se desprenden las épicas, funcionalidades y casos de uso que deben implementarse. No obstante, se asume que el equipo cuenta con el criterio técnico y el sentido común necesarios para tomar decisiones de diseño e incorporar aquellos elementos que resulten evidentes o implícitos, aun cuando no estén explicitados de manera literal.

Cualquier decisión relevante, supuesto asumido o cambio respecto del alcance original deberá ser debidamente comunicado y consensuado con el team leader antes de su implementación.

Requisitos

- La aplicación principal deberá ser exclusivamente mobile y estará orientada a los usuarios finales.
- El backoffice deberá implementarse como una aplicación web, de uso exclusivo para administradores.
- La solución deberá diseñarse bajo una arquitectura orientada a microservicios, garantizando desacoplamiento, escalabilidad y mantenibilidad.
- La aplicación debe contar con persistencia y el uso de, al menos, dos bases de datos.
- El backend no puede ser desarrollado en una única tecnología (ej. no puede estar todo en Python).
- La aplicación deberá estar desplegada en un entorno productivo en la nube. Cada microservicio deberá estar contenedorizado con Docker, asegurando portabilidad y consistencia entre ambientes.
- Se deberán proteger los datos de los usuarios, cumpliendo con buenas prácticas de seguridad y tomando como referencia los lineamientos del OWASP Top 10.
- El desarrollo deberá seguir un enfoque iterativo e incremental. Se espera la adopción de metodologías ágiles y herramientas adecuadas para la planificación, seguimiento y gestión del trabajo.
- El equipo será responsable tanto de la documentación técnica y funcional como de asegurar la calidad y mantenibilidad del código.
- Se deberán implementar distintos niveles de testing para garantizar la robustez del sistema, incluyendo pruebas unitarias, de integración y pruebas de estrés con carga de datos en volumen, a fin de validar todos los flujos críticos de la aplicación.
- Todos los servicios backend deberán incluir tests de integración, validando la correcta interacción entre componentes y dependencias externas.
- Cada microservicio, incluyendo el frontend, deberá contar con una cobertura mínima del 85% de tests unitarios. Estos tests deberán

ejecutarse automáticamente como parte del pipeline de Integración Continua (CI).
- Se deberá implementar un pipeline de Continuous Delivery (CD) utilizando GitHub Actions, automatizando el proceso de build, test y despliegue.
- La aplicación deberá contar con capacidades de observabilidad y monitoreo, permitiendo recolectar métricas, logs y trazas para detectar incidentes y analizar el comportamiento del sistema.
- Al menos uno de los microservicios deberá implementar un mecanismo de Rate Limiting, con el objetivo de controlar el tráfico y proteger el sistema ante abusos o picos de carga.

Inteligencia Artificial

A partir de la segunda mitad del semestre, el proyecto evolucionará con la incorporación de Historias de Usuario enfocadas en la integración de Inteligencia Artificial. Estas propuestas surgirán de la sinergia entre alumnos y tutores durante las reuniones semanales.
Cada grupo deberá implementar, al menos, una funcionalidad que integre IA, y deberá ser acordada con el tutor.

Condición de aprobación

- Cumplir con todos los Requisitos mencionados anteriormente.
- Realizar el total de las historias de usuario obligatorias, incluyendo todos sus criterios de aceptación.
- Implementar un total de 5 puntos de historias optativas por cada integrante del grupo, incluyendo todos sus criterios de aceptación.
- Proponer e implementar, al menos, una funcionalidad que utiliza Inteligencia Artificial.

Épicas

A continuación se detallan las historias de usuario que el sistema debe contemplar. Se diferencian en **obligatorias (⚡)** y **optativas (➕).**

E.1 Usuarios

H1. Registro de Usuarios ⚡

COMO usuario nuevo
QUIERO registrarme con mis datos personales y validar mi cuenta mediante un código de correo electrónico
PARA asegurar la autenticidad de mi perfil y acceder de forma segura al sistema.

- **CA.1:** Un usuario no debe poder entrar al sistema si su cuenta no fue validada por el token enviado al email.
- **CA.2:** El sistema debe validar que el formato del email sea correcto y no debe permitir correos duplicados.
- **CA.3:** El nombre de usuario debe ser único y contener entre 4 y 15 caracteres (solo letras y números).
- **CA.4:** La contraseña debe tener al menos 8 caracteres, una mayúscula y un número, y guardarse encriptada.
- **CA.5:** El sistema debe validar que no se envíen campos obligatorios vacíos o nulos desde el cliente.
- **CA.6:** El token o link de validación enviado al correo debe tener un tiempo de expiración (ej. 24 horas). Si expira, el usuario debe poder solicitar un reenvío desde la pantalla de login.
- **CA.7:** La validación de unicidad del email debe ser case-insensitive (es decir, Alumno@udesa.edu.ar es lo mismo que alumno@udesa.edu.ar).

H2. Inicio de Sesión ⚡

COMO usuario registrado
QUIERO ingresar mis credenciales (nombre de usuario/email y contraseña)
PARA acceder a mi cuenta personal y utilizar las funcionalidades del sistema.

- **CA.1:** El sistema debe proveer un token JWT con un tiempo de expiración definido al iniciar sesión correctamente.

- **CA.2:** Si el usuario falla la contraseña 5 veces consecutivas, la cuenta debe bloquearse temporalmente por 15 minutos.
- **CA.3:** Si el usuario o la contraseña son incorrectos, el mensaje de error debe ser genérico (ej. "Credenciales inválidas") para evitar enumeración de usuarios (no revelar si el que falló fue el email o la clave).
- **CA.4:** Si el usuario ingresa credenciales correctas pero su cuenta **no está validada** (falta paso del CA.1 de la H1), el sistema debe denegar el acceso y mostrar un mensaje invitándolo a revisar su casilla de correo.
- **CA.5:** El sistema debe impedir el login y mostrar un mensaje de "Cuenta suspendida" si el usuario fue bloqueado por un Administrador o si eliminó su cuenta (soft-delete).

H3. Cierre de Sesión ⚡

COMO usuario registrado
QUIERO cerrar mi sesión y que el token de acceso sea invalidado
PARA garantizar que nadie más use la app desde mi dispositivo.

- **CA.1:** El token de sesión activo debe ser revocado en el backend.
- **CA.2:** En la app mobile, se deben borrar de forma segura los datos relacionados a la sesión local y el JWT del usuario.

H4. Eliminación de Cuenta ⚡

COMO usuario registrado
QUIERO poder eliminar mi cuenta permanentemente
PARA que se borre toda mi información (perfil, amistades e historial de ubicaciones).

- **CA.1:** Se debe implementar un "soft-delete" en la base de datos manteniendo la integridad referencial, pero el usuario no debe poder volver a loguearse.
- **CA.2:** Todas las relaciones de amistad del usuario eliminado deben eliminarse automáticamente y desaparecer de las listas de sus amigos.

- **CA.3:** Para confirmar la eliminación definitiva, la app móvil debe requerir que el usuario reingrese su contraseña actual (evitar borrados accidentales o maliciosos si el celular quedó desbloqueado).
- **CA.4:** Todas las solicitudes de amistad pendientes (tanto las enviadas como las recibidas) deben ser eliminadas lógicamente o canceladas para no generar errores en las vistas de los otros usuarios.
- **CA.5:** El último pin de ubicación y el historial de ubicaciones del usuario eliminado deben dejar de renderizarse en los mapas de todos sus amigos al instante.

H5. Olvidé Mi Contraseña ⚡

COMO usuario registrado
QUIERO solicitar el restablecimiento de mi contraseña ingresando mi email o nombre de usuario
PARA recuperar el acceso a mi cuenta en caso de olvido.

- **CA.1:** El usuario debe recibir un link con una duración máxima de 10 minutos.
- **CA.2:** Si el link expira, la interfaz debe mostrar un error claro y dar la opción de enviar uno nuevo.
- **CA.3:** Al ingresar la nueva contraseña, esta debe validarse (confirmación doble) y cumplir con las mismas políticas de seguridad de la H1.
- **CA.4:** Al solicitar el restablecimiento, el mensaje devuelto debe ser siempre genérico, sin revelar si la cuenta realmente está registrada, para evitar ataques de enumeración de usuarios.
- **CA.5:** El link debe ser de un solo uso, una vez utilizado con éxito para cambiar la contraseña, debe ser invalidado.
- **CA.6:** La nueva contraseña ingresada por el usuario no debe ser igual a su contraseña actual.
- **CA.7:** Al cambiar la contraseña con éxito, el sistema debe revocar automáticamente todos los tokens de sesión (JWT) activos, forzando a que vuelva a iniciar sesión con su nueva clave.

- **CA.8:** Se debe limitar la cantidad de veces que se puede pedir un token de recuperación para un mismo correo.

H6. Editar mi perfil ⚡

COMO usuario registrado
QUIERO editar los campos de datos de mi perfil
PARA mantener mi información actualizada.

- **CA.1:** El sistema debe validar que los campos de texto no superen los caracteres máximos permitidos (ej. Biografía max 150 caracteres).
- **CA.2:** No se puede cambiar el email.
- **CA.3:** El usuario debe poder establecer una biografía.
- **CA.4:** El backend debe sanitizar estrictamente los campos de texto eliminando o escapando etiquetas HTML/Scripts para prevenir ataques de inyección.
- **CA.5:** El nombre de usuario es un campo obligatorio y no puede quedar vacío, ni contener únicamente espacios en blanco.
- **CA.6:** Al guardar los cambios exitosamente, la aplicación móvil debe actualizar el estado global y reflejar inmediatamente los nuevos datos en la pantalla.

H7. Preferencias ⚡

COMO usuario registrado
QUIERO configurar mis preferencias
PARA que la aplicación se adapte a mis necesidades de privacidad y rendimiento.

- **CA.1:** El usuario debe poder establecer el rango de búsqueda en un radio de 1 a 50 kilómetros.
- **CA.2:** El usuario debe poder elegir la frecuencia de actualización de su ubicación (ej. cada 5, 15 o 30 minutos).
- **CA.3:** El backend debe rechazar cualquier intento de guardar un rango de búsqueda fuera de los límites permitidos (menor a 1 km o mayor a 50 km) o con formatos inválidos (letras, nulos).

- **CA.4:** El backend solo debe aceptar los valores exactos definidos para la frecuencia (ej. 5, 15, 30). Si un usuario malintencionado intenta enviar 1 minuto a través de la API para saturar el servidor de ubicaciones, debe ser rechazado.
- **CA.5:** Al crearse una cuenta nueva (durante el Registro), el sistema debe asignar automáticamente preferencias por defecto.

H8. Foto de Perfil ➕

COMO usuario registrado
QUIERO subir y actualizar una foto de perfil
PARA que mis amigos puedan reconocerme visualmente en el mapa.

- **CA.1:** El sistema solo debe aceptar formatos válidos de imagen (JPG, PNG).
- **CA.2:** La imagen no debe superar los 5MB.
- **CA.3:** El backend debe validar el contenido real del archivo (inspeccionando el MIME type o magic numbers) y no confiar únicamente en la extensión del nombre del archivo, previniendo así la subida encubierta de scripts maliciosos o ejecutables.
- **CA.4:** Al actualizar la foto de perfil, el backend debe eliminar el archivo de la foto anterior del storage.
- **CA.5:** Si el usuario es nuevo, no subió ninguna foto, o decide eliminar la actual, el sistema debe proveer y renderizar un "avatar por defecto" (ej. una silueta genérica o las iniciales de su nombre) tanto en la vista de perfil como en los pines del mapa.
- **CA.6:** El usuario debe contar con un botón explícito de "Eliminar foto actual", el cual ejecutará el CA.4 y devolverá el perfil al estado del CA.5.
- **CA.7:** El backend debe procesar la subida mediante streams directamente hacia el storage. El archivo completo no debe ser cargado en la memoria del servidor en ningún momento del proceso.

H9. Social Login ➕

COMO usuario nuevo o registrado

QUIERO iniciar sesión utilizando mi cuenta de Google o Apple
PARA acceder más rápido sin tener que recordar una nueva contraseña.

- **CA.1:** Si el email de Google/Apple no existe, se debe crear la cuenta y autocompletar el nombre.
- **CA.2:** Si el email ya existe, se debe vincular el método de inicio de sesión a la cuenta existente.
- **CA.3:** La validación final del inicio de sesión debe hacerse siempre en el backend. El servidor debe recibir el token (ID Token) emitido por Google/Apple desde la app móvil y verificar su firma y validez directamente con los servidores del proveedor antes de emitir el JWT propio del sistema.
- **CA.4:** Si una cuenta fue creada exclusivamente mediante Social Login y el usuario intenta usar la función "Olvidé mi contraseña" (H5) o loguearse con email/contraseña estándar, el sistema debe indicarle que su cuenta está vinculada a un proveedor social y debe usar ese botón para ingresar.

H10. Tema de la Aplicación ➕

COMO usuario registrado
QUIERO poder alternar entre un tema "Claro" y un tema "Oscuro"
PARA que la interfaz sea más cómoda de usar según la iluminación de mi entorno.

- **CA.1:** Esta preferencia debe guardarse únicamente de forma local en el dispositivo del usuario.
- **CA.2:** La aplicación debe cambiar los colores de fondo y texto de forma inmediata al seleccionar la opción, sin necesidad de reiniciar la app.

H11. Enviar Feedback o Reportar Error ➕

COMO usuario registrado
QUIERO un formulario simple dentro de Configuración para enviar comentarios o reportar un bug
PARA ayudar a mejorar la aplicación si algo falla.

- **CA.1:** El formulario debe incluir un campo de texto libre (máx. 500 caracteres).
- **CA.2:** El envío del formulario debe generar un registro y enviar un email a la cuenta de soporte del equipo.
- **CA.3:** Al enviar el formulario, la aplicación móvil debe adjuntar automáticamente y de forma oculta la metadata del dispositivo: Sistema Operativo (iOS/Android), versión de la app, e ID del usuario.
- **CA.4:** El endpoint que recibe el feedback debe estar protegido por un Rate Limit estricto (ej. máximo 2 envíos por usuario por hora) para evitar que inunden el Backoffice o el correo de soporte con spam.
- **CA.5:** El sistema (tanto frontend como backend) no debe permitir el envío de formularios donde el campo de texto esté vacío o contenga únicamente espacios en blanco.

H12. Aceptación de Términos y Política de Privacidad ⚡

COMO usuario nuevo
QUIERO leer y aceptar los Términos y Condiciones y la Política de Privacidad antes de registrarme
PARA saber legalmente cómo se utilizarán y almacenarán mis datos de ubicación.

- **CA.1:** El formulario de registro (H1) debe incluir un *checkbox* obligatorio con enlaces a textos estáticos de Términos y Privacidad. Si no está marcado, el botón de registrarse debe estar deshabilitado.
- **CA.2:** El backend debe guardar un registro (booleano y *timestamp*) indicando cuándo el usuario aceptó estas políticas.

H13. Cambiar Contraseña ⚡

COMO usuario registrado y logueado
QUIERO poder cambiar mi contraseña desde la pantalla de configuración ingresando mi clave actual
PARA mantener mi cuenta segura de forma proactiva sin necesidad de usar el flujo de recuperación por email.

- **CA.1:** El formulario debe solicitar la "Contraseña actual", "Nueva contraseña" y "Confirmar nueva contraseña".
- **CA.2:** El backend debe validar que la nueva contraseña cumpla con los requisitos de seguridad (H1) y no sea igual a la anterior.
- **CA.3:** Al cambiar la contraseña exitosamente, el sistema debe invalidar/revocar todos los tokens JWT activos del usuario (incluyendo la sesión actual) y redirigir automáticamente a la app móvil a la pantalla de Inicio de Sesión.
- **CA.4:** El backend debe validar que la "Contraseña actual" ingresada sea correcta. Si el usuario falla este paso 3 veces consecutivas, la cuenta debe bloquearse temporalmente por 15 minutos (reutilizando la lógica de la H2).
- **CA.5:** El sistema debe enviar un correo electrónico automático al usuario notificando que su contraseña ha sido modificada, recomendando contactar a soporte si él no realizó el cambio.

H14. Onboarding de Permisos Móviles ➕

COMO usuario que acaba de iniciar sesión por primera vez
QUIERO ver pantallas explicativas (onboarding) sobre por qué la app necesita mi ubicación en segundo plano y enviarme notificaciones
PARA entender el valor de la app antes de que el sistema operativo me lance los pop-ups nativos de permisos.

- **CA.1:** El usuario debe visualizar un carrusel explicativo antes de que la aplicación ejecute las funciones nativas de solicitud de permisos de iOS/Android.
- **CA.2:** Si el usuario deniega el permiso de GPS "Siempre" desde el sistema operativo, la app debe mostrar un Empty State bloqueante en la pantalla principal indicando que no puede funcionar sin la ubicación, y brindando un botón que lo redirija directamente a la configuración del celular.

E.2 Amistades

H1. Agregar nuevo amigo ⚡

COMO usuario del sistema
QUIERO enviar una invitación de amistad buscando a otros por su username
PARA expandir mi red dentro de la plataforma.

- **CA.1:** No se puede enviar una solicitud a un usuario que ya es amigo, que ya tiene una solicitud pendiente, o a uno mismo.
- **CA.2:** La relación se guarda en estado "Pendiente" hasta la respuesta del receptor.
- **CA.3:** Si el Usuario A le envía una solicitud al Usuario B, pero el Usuario B ya le había enviado una solicitud previamente al Usuario A (que seguía pendiente), el sistema debe aceptar automáticamente la amistad y cambiar el estado a "Aceptado" para ambos, sin generar un error de duplicidad.
- **CA.4:** El sistema debe impedir enviar una solicitud si el usuario receptor tiene bloqueado al emisor. Para evitar revelar que fue bloqueado, el backend debe devolver un mensaje genérico de éxito ("Solicitud enviada") pero no crear el registro en la base de datos ni notificar al receptor.
- **CA.5:** Para evitar el envío masivo de spam, un usuario no puede enviar más de 20 solicitudes de amistad por hora.

H2. Listado de Solicitudes Pendientes ⚡

COMO usuario receptor de una invitación
QUIERO visualizar mi lista de solicitudes pendientes
PARA aceptarlas o rechazarlas.

- **CA.1:** Al aceptar, el estado cambia a "Aceptado" y ambos usuarios pueden ver sus ubicaciones (según preferencias).
- **CA.2:** Al rechazar, la solicitud se elimina lógicamente de la base de datos sin notificar al emisor.

- **CA.3:** Las solicitudes pendientes deben mostrarse ordenadas cronológicamente de forma descendente.
- **CA.4:** Si el emisor de la solicitud elimina su cuenta o es suspendido por un administrador antes de que el receptor responda, la solicitud debe desaparecer automáticamente de esta lista.
- **CA.5:** Si la lista de solicitudes supera los 20 registros, el backend debe paginar los resultados y la app móvil debe implementar scroll infinito.

H3. Eliminar amigo ⚡

COMO usuario del sistema
QUIERO poder borrar a un amigo
PARA dejar de compartir mi ubicación con él.

- **CA.1:** El sistema debe pedir una confirmación emergente antes de eliminar al amigo.
- **CA.2:** Al eliminarse la amistad, el usuario debe desaparecer inmediatamente del mapa del otro usuario.
- **CA.3:** La eliminación de la amistad debe ser simétrica. Al borrar la relación, el Usuario A desaparece de la lista del Usuario B, y el Usuario B de la lista del Usuario A simultáneamente.
- **CA.4:** Si el usuario eliminado pertenecía a una "Lista de mejores amigos" (H12) o a alguna configuración de notificaciones específica, el backend debe limpiar automáticamente esos registros huérfanos para evitar inconsistencias.
- **CA.5:** Al confirmar la eliminación en la app móvil, el cambio debe reflejarse en tiempo real en la Lista de Amigos local.

H4. Modo Fantasma ➕

COMO usuario del sistema
QUIERO activar un “Modo Fantasma”
PARA ocultar mi ubicación temporalmente (24 horas por default) a todos mis contactos.

- **CA.1:** El sistema debe dejar de emitir y registrar ubicaciones del usuario en el backend mientras el modo esté activo.
- **CA.2:** Pasado el tiempo en modo fantasma, el sistema debe desactivar automáticamente el modo y reanudar el tracking.
- **CA.3:** Mientras el modo esté activo, el usuario debe ver un indicador visual claro y persistente en su propia pantalla que le recuerde que no está compartiendo su ubicación y le muestre el tiempo restante.
- **CA.4:** El usuario debe tener un botón accesible para desactivar el "Modo Fantasma" de forma manual en cualquier momento.

H5. Modo Privado ⚡

COMO usuario del sistema
QUIERO poder poner mi perfil en modo privado
PARA que mi cuenta no aparezca en las búsquedas globales de descubrimiento.

- **CA.1:** Si el modo privado está activo, el usuario solo compartirá su ubicación con sus amigos.
- **CA.2:** El usuario tendrá la opción de desactivar el modo privado, lo que significa que su usuario pasará a ser público nuevamente.
- **CA.3:** El cambio de estado (Público a Privado o viceversa) debe impactar de forma inmediata.

H6. Descubrimiento ⚡

COMO usuario del sistema
QUIERO ver un radar de usuarios cercanos que no sean mis amigos
PARA enviarles invitaciones basadas en cercanía.

- **CA.1:** Solo deben mostrarse usuarios cuyo "Modo Privado" o "Modo Fantasma" estén desactivados.
- **CA.2:** El radar debe respetar el límite de kilómetros configurado en las preferencias del usuario.

H7. Visualización de Lista de Amigos ⚡

COMO usuario del sistema
QUIERO acceder a un listado con todos mis amigos confirmados
PARA ver con quiénes estoy compartiendo mi información.

- **CA.1:** La lista debe estar ordenada alfabéticamente o por cercanía actual (intercambiable).
- **CA.2:** La lista debe contar con paginación o scroll infinito si supera los 20 amigos.
- **CA.3:** Si el usuario no tiene amigos confirmados, la pantalla debe mostrar un diseño amigable con un botón grande que lo redirija a la pantalla de "Descubrimiento" o "Agregar amigo".

H8. Bloquear Usuario ⚡

COMO usuario del sistema
QUIERO bloquear a otro usuario
PARA que no pueda ver mi ubicación, enviarme mensajes ni buscarme en la lista de usuarios.

- **CA.1:** El usuario bloqueado no recibe notificación del bloqueo.
- **CA.2:** El usuario que bloquea debe tener una sección en sus ajustes para ver la "Lista de bloqueados" y poder desbloquearlos.
- **CA.3:** Al bloquear a un usuario, si existía una relación de amistad ("Aceptada" o "Pendiente"), el sistema debe eliminarla o romperla automáticamente.
- **CA.4:** El usuario bloqueado no debe poder ver el perfil del bloqueador; si intenta acceder mediante un link directo o historial, el backend debe devolver un error 404.
- **CA.5:** Si existía un historial de chat entre ambos, el hilo de conversación debe quedar deshabilitado (read-only) y no se debe permitir el envío de nuevos mensajes.

H9. Denunciar Usuario ➕

COMO usuario del sistema

QUIERO poder reportar a un usuario por comportamiento indebido
PARA mantener la seguridad en la comunidad.

- **CA.1:** El sistema debe permitir elegir un motivo y enviar el reporte al backoffice.
- **CA.2:** Si un usuario recibe más de 5 reportes de cuentas distintas, su cuenta pasa temporalmente a estado "En revisión".
- **CA.3:** El sistema debe impedir que un mismo usuario reporte a la misma persona más de una vez en un lapso de 24 horas.
- **CA.4:** Si se cumple la condición de 5 reportes y la cuenta pasa a estado "En revisión", el backend debe revocar inmediatamente todos los tokens JWT activos de ese usuario, expulsándolo de la app hasta que un administrador resuelva el caso.

H10. Mensajes entre Amigos ➕

COMO usuario del sistema
QUIERO acceder a una sección de mensajería
PARA conversar en privado con mis amigos.

- **CA.1:** Solo se pueden enviar mensajes a usuarios con estado de amistad "Aceptado".
- **CA.2:** Los mensajes deben guardarse en la base de datos para que el historial esté disponible si el usuario cierra la aplicación y la vuelve a abrir.
- **CA.3:** Si un usuario envía un mensaje y el receptor no está conectado, el backend debe guardar el mensaje y enviarlo como una notificación push.
- **CA.4:** Al abrir un chat, el sistema debe cargar solo los últimos 50 mensajes, implementando paginación (scroll hacia arriba) para cargar el historial anterior.

H11. Amigos en línea ➕

COMO usuario del sistema
QUIERO ver quiénes de mis amigos están conectados

PARA saber quién está usando la app en el momento.

- **CA.1:** Un usuario se considera "en línea" si interactuó con la app en los últimos 5 minutos.
- **CA.2:** Debe mostrarse un indicador visual (ej. punto verde) en la foto de perfil del listado.
- **CA.3:** Si el usuario tiene activado el "Modo Fantasma" (H4), debe aparecer siempre como desconectado (sin el punto verde) para sus amigos, independientemente de si está usando la app o no.

H12. Lista de mejores amigos ➕

COMO usuario del sistema
QUIERO crear listas personalizadas (ej. “Grupo de Estudio”)
PARA compartir mi ubicación solo con ese subconjunto de personas.

- **CA.1:** Un amigo puede pertenecer a más de una lista.
- **CA.2:** Al activar el envío de ubicación, se debe poder seleccionar "Todas" o una "Lista específica".
- **CA.3:** Si un usuario elimina a un amigo de su red (H3), el backend debe sacarlo automáticamente de cualquier lista personalizada a la que perteneciera.
- **CA.4:** Si el usuario elimina una lista personalizada completa que estaba configurada como "activa" para compartir ubicación, la configuración debe volver automáticamente al valor por defecto ("Todos mis amigos").

H13. Invitar Usuarios Externos ➕

COMO Usuario del sistema
QUIERO enviar una invitación por email o link a externos
PARA que se sumen a la aplicación.

- **CA.1:** El link generado debe incluir un parámetro de "referido" para asociar al usuario nuevo con quien lo invitó (deep linking).
- **CA.2:** Cuando el usuario invitado se registra exitosamente utilizando el link de referido, el sistema debe generar automáticamente una relación

de amistad "Aceptada" entre el nuevo usuario y quien lo invitó, saltándose el paso de enviar solicitud.
- **CA.3:** Un usuario no puede generar más de 10 links de invitación únicos por día.

H14. Visualización de Perfil de Amigo ⚡

COMO usuario del sistema
QUIERO entrar al perfil de un amigo para ver su biografía, su estado actual y su historial de lugares recientes
PARA conocer más sobre su actividad y por dónde estuvo.

- **CA.1:** El perfil solo debe mostrar el historial de lugares si ambos usuarios tienen una relación de amistad.
- **CA.2:** La pantalla debe renderizar la biografía del usuario y debajo una lista ordenada cronológicamente (del más reciente al más antiguo) de sus últimas ubicaciones o "check-ins".
- **CA.3:** Si el amigo tiene activado su "Modo Privado" o está en "Modo Fantasma", la lista cronológica de ubicaciones debe ocultarse y mostrarse un mensaje de "El historial de este usuario es privado".
- **CA.4:** Para optimizar la respuesta, el endpoint solo debe devolver los últimos 10 check-ins o ubicaciones.

E.3 Ubicaciones

H1. Enviar Ubicación ⚡

COMO usuario del sistema
QUIERO enviar mis coordenadas (Latitud/Longitud) al sistema según la frecuencia configurada
PARA que se mantenga mi posición actualizada.

- **CA.1:** La app debe solicitar y manejar correctamente los permisos de ubicación nativos del SO (iOS/Android).
- **CA.2:** Se debe validar que las coordenadas tengan un formato válido antes de guardarlas.

- **CA.3:** La app móvil debe detener el envío automático de ubicaciones en segundo plano si la batería del dispositivo cae por debajo del 20%.
- **CA.4:** El backend debe rechazar peticiones si el usuario intenta enviar su ubicación con una frecuencia mayor a la configurada (ej. si configuró 15 minutos, y manda 5 requests en 1 minuto, devolver HTTP 429 Too Many Requests).
-

H2. Listado de Ubicaciones ⚡

COMO usuario del sistema
QUIERO ver dónde se encuentran mis amigos en tiempo real
PARA visualizar las coordenadas traducidas a distancias estimadas.

- **CA.1:** El sistema debe calcular y mostrar la distancia en km o metros entre el usuario actual y sus amigos.
- **CA.2:** La información debe estar protegida para que un usuario no pueda consultar por API la ubicación de alguien que no sea su amigo.

H3. Visualización en Mapa Interactivo ⚡

COMO usuario del sistema
QUIERO ver un mapa interactivo con pines de mis amigos
PARA entender visualmente dónde se encuentran.

- **CA.1:** El mapa debe agrupar los pines si hay varios amigos muy juntos, para evitar superposiciones en el celular.
- **CA.2:** Al tocar un pin, se debe mostrar el nombre, la foto y hace cuántos minutos se actualizó esa ubicación.
- **CA.3:** La pantalla del mapa debe contar con un botón de "Centrar en mí", que devuelva la cámara a la latitud y longitud actuales del dispositivo local.

H4. Sincronización Offline ➕

COMO usuario de la app móvil
QUIERO que la app guarde mi ruta si pierdo conexión a internet

PARA enviarla al servidor cuando vuelva a tener datos.

- **CA.1:** La app debe guardar las coordenadas con su timestamp en una base de datos local (ej. SQLite).
- **CA.2:** Al recuperar conexión, debe enviar un batch de ubicaciones al backend y limpiar la tabla local.
- **CA.3:** La base de datos local debe guardar un máximo de 100 puntos de ubicación o descartar aquellos con más de 24 horas de antigüedad, para no saturar el almacenamiento del celular.

H5. Establecer Punto de Encuentro ➕

COMO usuario del sistema
QUIERO marcar un punto de interés temporal en el mapa y compartirlo
PARA coordinar una reunión sin mandar mi ubicación en vivo.

- **CA.1:** El creador del punto debe poder ponerle un nombre (ej. "Facultad") y seleccionar a qué amigos notificará.
- **CA.2:** El punto de encuentro debe desaparecer automáticamente del mapa pasadas 12 horas desde su creación.
- **CA.3:** El backend debe implementar un mecanismo que elimine físicamente o desactive los puntos de encuentro una vez transcurridas las 12 horas.
- **CA.4:** Al crearse el punto, los amigos seleccionados deben recibir una notificación Push automática invitándolos a ver el mapa.
- **CA.5:** Un usuario solo puede tener un punto de encuentro activo a la vez. Si crea uno nuevo, el anterior se desactiva.

H6. Historial de Ubicaciones➕

COMO usuario del sistema
QUIERO ver la ruta reciente (ej. últimas 4 horas) de un amigo si él lo permite
PARA saber el recorrido de mi amigo hacia un punto.

- **CA.1:** El amigo debe tener explícitamente activada la opción "Permitir historial de ruta" en sus preferencias.

- **CA.2:** El backend solo debe retornar puntos de ubicación con un timestamp menor a 4 horas de antigüedad.
- **CA.3:** Para que la app móvil no colapse al dibujar la ruta, el backend debe devolver un máximo de 50 puntos interpolados dentro de esas 4 horas.
- **CA.4:** El microservicio debe validar que el usuario que consulta tenga una relación de amistad "Aceptada" vigente con el usuario consultado antes de devolver la ruta.

H7. Etiqueta de Lugar Manual ⚡

COMO usuario del sistema
QUIERO escribir manualmente un texto breve indicando dónde estoy (ej. "En la facu", "Casa de Juan")
PARA darle contexto a mi ubicación en el mapa sin depender de coordenadas exactas.

- **CA.1:** El campo de texto para la etiqueta no debe superar los 30 caracteres.
- **CA.2:** Esta etiqueta debe renderizarse en la app móvil justo debajo del nombre de usuario al tocar su pin en el mapa.
- **CA.3:** El usuario debe contar con un botón de "Borrar etiqueta" que envíe un valor nulo o vacío al backend para limpiar el estado.
- **CA.4:** Si el usuario se aleja más de 500 metros del punto donde creó la etiqueta manual (o si pasan más de 6 horas), el sistema debe borrar o invalidar la etiqueta automáticamente para que no quede información desactualizada.
- **CA.5:** Al igual que la biografía, el texto ingresado debe ser sanitizado para evitar ataques.

H8. "Me gusta" a la Ubicación ➕

COMO Usuario del sistema
QUIERO darle un "Me gusta" (botón de corazón) al pin de un amigo en el mapa
PARA hacerle saber que vi dónde está de forma amigable.

- **CA.1:** El sistema solo debe permitir enviar un "Me gusta" cada vez que el amigo actualiza sus coordenadas (si el amigo se mueve y el sistema registra una nueva ubicación, se habilita nuevamente el botón).
- **CA.2:** El receptor debe recibir una notificación simple in-app o push con el texto "[Nombre del amigo] le dio Me gusta a tu ubicación".

H9. Color del Pin en el Mapa ➕

COMO Usuario registrado
QUIERO elegir el color de mi pin (marcador) en el mapa
PARA diferenciarme visualmente de mis amigos de forma rápida.

- **CA.1:** El sistema debe ofrecer una paleta estática predefinida de 5 colores en la pantalla de preferencias.
- **CA.2:** Al cambiar el color, este debe guardarse en la base de datos y actualizarse en el mapa de mis amigos la próxima vez que consulten mi ubicación.

H10. Zonas Seguras / Zonas de Privacidad ➕

COMO usuario del sistema
QUIERO poder establecer un radio en el mapa (ej. mi casa o mi trabajo) como "Zona Segura"
PARA que, al entrar en esa área geográfica, mi ubicación exacta se congele o se oculte automáticamente para mis amigos.

- **CA.1:** El usuario debe poder definir hasta 2 zonas seguras mediante un pin en el mapa y un selector de radio.
- **CA.2:** Cuando el backend reciba coordenadas que caigan dentro de esa zona, la API de listado (H2) debe devolver a los amigos un estado de "En zona segura" ocultando las coordenadas exactas, o bien, devolver la última coordenada registrada antes de entrar a la zona.

E.4 Notificaciones

H1. Recibir notificaciones de amistad ⚡

COMO usuario del sistema
QUIERO recibir una notificación push cuando alguien me envíe una solicitud de amistad
PARA aceptarlos rápidamente.

- **CA.1:** El servicio de notificaciones debe usar FCM (Firebase Cloud Messaging) para entregar las notificaciones push.
- **CA.2:** Al tocar la notificación, la app debe abrirse directamente en la pantalla de "Solicitudes Pendientes".
- **CA.3:** El backend debe almacenar y actualizar el "Device Token" (FCM) del usuario cada vez que inicia sesión, ya que estos tokens pueden rotar o expirar.
- **CA.4:** Si el servidor de Firebase responde que un token ya no es válido, el backend debe borrar ese token de la base de datos para no enviar peticiones muertas.

H2. Notificación de solicitud aceptada ⚡

COMO usuario del sistema
QUIERO recibir una notificación cuando alguien acepte mi invitación
PARA saber que ya podemos interactuar en el mapa.

- **CA.1:** El mensaje de la notificación debe incluir el nombre de usuario que aceptó la solicitud.
- **CA.2:** Al tocar la notificación push con la app cerrada o en segundo plano, la aplicación debe abrirse y redirigir al usuario directamente al perfil del nuevo amigo o centrar el mapa en su ubicación actual.

H3. Centro de Notificaciones In-App ⚡

COMO usuario del sistema
QUIERO tener un panel interno (campanita) con el historial de notificaciones
PARA revisar alertas que pude haber descartado en mi celular.

- **CA.1:** Las notificaciones no leídas deben tener un indicador visual distinto a las ya leídas.
- **CA.2:** El usuario debe poder marcar todas como leídas con un solo botón.
- **CA.3:** El backend debe devolver un máximo de 20 notificaciones por página ordenadas de la más reciente a la más antigua.
- **CA.4:** El usuario debe poder deslizar hacia un lado (swipe) o presionar un ícono de tacho de basura para borrar una notificación particular.
- **CA.5:** La notificación debe eliminarse lógicamente en la base de datos.

H4. Notificación por proximidad ➕

COMO usuario de la plataforma
QUIERO recibir una alerta push cuando un amigo entre en mi radio configurado
PARA poder proponerle un encuentro casual.

- **CA.1:** El backend debe calcular las distancias en background y disparar el evento solo 1 vez (evitando spam si el amigo entra y sale del borde del radio constantemente).
- **CA.2:** Esta opción debe poder apagarse por completo desde Preferencias.
- **CA.3:** Una vez disparada la notificación de proximidad para un amigo específico, el sistema debe iniciar un período de cooldown de al menos 2 horas antes de volver a notificar por ese mismo amigo, aunque entre y salga del radio repetidas veces.

H5. Configurar Preferencias de Notificación ➕

COMO usuario del sistema
QUIERO configurar qué tipos de notificaciones quiero recibir y cuáles no
PARA evitar notificaciones que no me interesan.

- **CA.1:** Debe existir un toggle individual por amigo en la lista de amistades para silenciar sus notificaciones de proximidad.

- **CA.2:** Se debe consultar obligatoriamente esta tabla de preferencias antes de enviar el push.

E.5 Administradores (Backoffice Web)

H1. Creación de Administradores ⚡

COMO administrador principal (SuperAdmin)
QUIERO poder crear nuevas credenciales de acceso para el backoffice
PARA delegar tareas de soporte a mi equipo.

- **CA.1:** Las contraseñas de los administradores deben crearse desde el panel, pero el nuevo administrador debe ser forzado a cambiarla en su primer login.
- **CA.2:** Debe existir un control de Roles (SuperAdmin, Moderador), donde el Moderador no pueda crear a otros admins.
- **CA.3:** La contraseña temporal generada/asignada debe expirar en 24 horas. Si el nuevo administrador no inicia sesión para cambiarla en ese lapso, el SuperAdmin debe generar una nueva.
- **CA.4:** El sistema debe validar que el correo ingresado pertenezca a un dominio autorizado (ej. @udesa.edu.ar) si se define una restricción institucional.

H2. Inicio de sesión como Administrador ⚡

COMO administrador del sistema
QUIERO iniciar sesión en el portal web del backoffice utilizando mi email y contraseña
PARA acceder a las herramientas de gestión y moderación.

- **CA.1:** El token JWT emitido por este endpoint debe contener el "Rol" del usuario (Admin/Moderador) para que el frontend del backoffice oculte o muestre las pantallas correspondientes.
- **CA.2:** Un usuario regular de la app móvil (Estudiante) NO debe poder iniciar sesión en el backoffice web con sus credenciales. El backend debe devolver un error 403 Forbidden.

- **CA.3:** El formulario de login del backoffice debe bloquearse por 30 minutos tras 3 intentos fallidos consecutivos, para mitigar ataques de fuerza bruta.

H3. Visualización de Métricas ⚡

COMO administrador del sistema
QUIERO visualizar un dashboard con métricas clave (usuarios totales, usuarios online, altas del mes)
PARA monitorear la salud y crecimiento de la plataforma.

- **CA.1:** El dashboard debe actualizarse en tiempo real o permitir refresco manual.
- **CA.2:** Debe mostrarse un gráfico de barras o líneas con los registros de la última semana.

H4. Buscador y Detalles de Usuarios ⚡

COMO administrador del sistema
QUIERO buscar usuarios por email o username y ver el detalle de su perfil
PARA dar soporte o auditar cuentas.

- **CA.1:** El panel de detalle debe mostrar estado (Activo, Bloqueado), fecha de registro, y cantidad de amigos.
- **CA.2:** La búsqueda debe soportar coincidencias parciales.
- **CA.3:** Debe implementar paginación.

H5. Bloqueo de Usuarios por Admin ⚡

COMO administrador del sistema
QUIERO cambiar el estado de un usuario a "Bloqueado"
PARA sancionar a quienes incumplan los términos y condiciones.

- **CA.1:** El administrador debe ingresar obligatoriamente un motivo en texto antes de ejecutar el bloqueo.
- **CA.2:** El usuario bloqueado debe perder el acceso mediante invalidación de token de forma inmediata (forzar cierre de sesión).

H6. Logs de Auditoría ➕

COMO administrador del sistema
QUIERO ver un registro de las acciones que hacen los otros administradores (quién bloqueó a quién y cuándo)
PARA tener control y trazabilidad de los cambios sensibles.

- **CA.1:** El sistema debe registrar Email del Admin, Acción (Ej. BLOQUEO_USUARIO), Entidad Afectada (ID de usuario) y Timestamp.
- **CA.2:** Los logs deben ser de "solo lectura" y ningún administrador debe poder borrarlos.
- **CA.3:** Los registros deben mostrarse paginados y ordenados por fecha descendente.

H7. Gestión de Denuncias ➕

COMO administrador del sistema
QUIERO visualizar una lista de denuncias realizadas por los usuarios
PARA revisar el caso y tomar acciones disciplinarias.

- **CA.1:** La lista debe ordenar los reportes por severidad (cantidad de reportes que recibió el mismo usuario).
- **CA.2:** El administrador debe tener botones rápidos de "Descartar denuncia" o "Suspender cuenta".
- **CA.3:** El administrador debe tener un botón para "Resolver/Cerrar caso", lo cual cambiará el estado de la denuncia a “Resuelta” y ocultará el reporte de la vista principal de casos pendientes.

H8. Exportar Datos ➕

COMO administrador del sistema
QUIERO poder descargar el listado de usuarios filtrados en formato CSV
PARA hacer análisis cruzados en Excel o herramientas externas.

- **CA.1:** El CSV debe generarse correctamente con las columnas de ID, Username, Email, Estado y Fecha de Registro.

- **CA.2:** Esta funcionalidad debe estar restringida exclusivamente al rol "SuperAdmin". Si un Moderador intenta invocar el endpoint, debe recibir un error HTTP 403 Forbidden.

H9. Registro de Última Conexión ⚡

COMO administrador del sistema
QUIERO ver la fecha y hora de la última vez que un usuario inició sesión
PARA identificar rápidamente cuentas inactivas, abandonadas o bots.

- **CA.1:** Cada vez que el endpoint de inicio de sesión responde exitosamente, el backend debe actualizar un campo fecha_ultimo_login en el registro del usuario.
- **CA.2:** Este dato debe mostrarse como una columna obligatoria en la tabla del Buscador de Usuarios del panel de administración.

H10. Borrado Forzado de Textos ➕

COMO administrador del sistema
QUIERO poder borrar el "Estado Corto" o la "Etiqueta de Lugar" de un usuario desde el panel web
PARA eliminar rápidamente textos que incumplan las normas (insultos, spam) sin tener que bloquear toda la cuenta.

- **CA.1:** El administrador debe tener un botón de acción llamado "Limpiar Textos" al visualizar el detalle de un usuario específico.
- **CA.2:** Al ejecutar la acción, el sistema debe sobreescribir los campos correspondientes con valores vacíos (strings vacíos o nulos) en la base de datos de manera inmediata.

H11. Estado de los Microservicios ⚡

COMO administrador del sistema
QUIERO visualizar un panel con el estado "En línea" o "Caído" de cada microservicio
PARA detectar rápidamente si hay una falla técnica en la plataforma.

- **CA.1:** El backoffice debe consumir un endpoint de healthcheck (/healthcheck) de cada microservicio.
- **CA.2:** Se debe mostrar un indicador visual simple (Verde = OK, Rojo = Caído) para cada servicio.
- **CA.3:** Si el endpoint /healthcheck de un microservicio tarda más de 5 segundos en responder, el backoffice web debe abortar la petición (Timeout) y marcar visualmente a ese servicio como "Caído" (Rojo) sin quedarse colgado esperando.
- **CA.4:** En caso de que un servicio se caiga, todos los administradores deben recibir una alerta por email.

H12. Gestión de Feedback/Reportes➕

COMO administrador del sistema
QUIERO visualizar una bandeja de entrada con los mensajes de feedback y reporte de errores (bugs) enviados por los usuarios desde la app móvil
PARA identificar problemas técnicos rápidamente y mejorar la plataforma.

- **CA.1:** La lista debe mostrar el texto enviado, el ID/nombre de usuario del usuario, la metadata técnica del dispositivo (iOS/Android) y la fecha de envío.
- **CA.2:** El administrador debe poder marcar los mensajes individuales como "Leídos" o "Resueltos" para sacarlos de la bandeja principal.
- **CA.3:** Si el usuario que envió el feedback es eliminado o bloqueado, el reporte debe conservarse en el backoffice (no debe borrarse en cascada) para mantener el registro del bug.

Puntos de Historias de Usuario

Épica / Historia de Usuario	Tipo	Puntos
E.1 Usuarios		
H1. Registro de Usuarios	⚡	3
H2. Inicio de Sesión	⚡	3
H3. Cierre de Sesión	⚡	1
H4. Eliminación de Cuenta	⚡	3
H5. Olvidé Mi Contraseña	⚡	3
H6. Editar mi perfil	⚡	2
H7. Preferencias	⚡	2
H8. Foto de Perfil	➕	3
H9. Social Login	➕	5
H10. Tema de la Aplicación	➕	1
H11. Enviar Feedback o Reportar Error	➕	3
H12. Aceptación de Términos y Política de Privacidad	⚡	1
H13. Cambiar Contraseña	⚡	3
H14. Onboarding de Permisos Móviles	➕	3
E.2 Amistades		
H1. Agregar nuevo amigo	⚡	3
H2. Listado de Solicitudes Pendientes	⚡	3
H3. Eliminar amigo	⚡	2
H4. Modo Fantasma	➕	2

H5. Modo Privado	⚡	2
H6. Descubrimiento	⚡	5
H7. Visualización de Lista de Amigos	⚡	3
H8. Bloquear Usuario	⚡	2
H9. Denunciar Usuario	➕	2
H10. Mensajes entre Amigos	➕	8
H11. Amigos en línea	➕	3
H12. Lista de mejores amigos	➕	3
H13. Invitar Usuarios Externos	➕	2
H14. Visualización de Perfil de Amigo	⚡	3
E.3 Ubicaciones		
H1. Enviar Ubicación	⚡	5
H2. Listado de Ubicaciones	⚡	3
H3. Visualización en Mapa Interactivo	⚡	5
H4. Sincronización Offline	➕	5
H5. Establecer Punto de Encuentro	➕	5
H6. Historial de Ubicaciones	➕	5
H7. Etiqueta de Lugar Manual	⚡	5
H8. "Me gusta" a la Ubicación	➕	2
H9. Color del Pin en el Mapa	➕	1
H10. Zonas Seguras / Zonas de Privacidad	➕	5
E.4 Notificaciones		

H1. Recibir notificaciones de amistad	⚡	2
H2. Notificación de solicitud aceptada	⚡	2
H3. Centro de Notificaciones In-App	⚡	3
H4. Notificación por proximidad	➕	5
H5. Configurar Preferencias de Notificación	➕	3
E.5 Administradores (Backoffice Web)		
H1. Creación de Administradores	⚡	5
H2. Inicio de sesión como Administrador	⚡	2
H3. Visualización de Métricas	⚡	5
H4. Buscador y Detalles de Usuarios	⚡	3
H5. Bloqueo de Usuarios por Admin	⚡	2
H6. Logs de Auditoría	➕	3
H7. Gestión de Denuncias	➕	3
H8. Exportar Datos	➕	2
H9. Registro de Última Conexión	⚡	2
H10. Borrado Forzado de Textos	➕	3
H11. Estado de los Microservicios	⚡	5
H12. Gestión de Feedback/Reportes	➕	3

Contrato Moral

La realización de este proyecto se basa en el compromiso mutuo de honestidad académica.

Se espera que cada estudiante sea el autor genuino de sus producciones, utilizando los recursos tecnológicos e inteligencias artificiales de manera ética: como un apoyo para la investigación y el enriquecimiento del contenido, y nunca como un sustituto del esfuerzo intelectual propio.